German University in Cairo
Department of Computer Science
Assoc. Prof. Mervat Abu El-Kheir

Architecture of Massively Scalable Applications, Spring 2026
Lab 4 Manual: Databases I (SQL)

Welcome to the Databases Lab! In this lab you will connect a Spring Boot application to PostgreSQL, map Java objects to database tables with JPA, model real relationships between entities, store semi-structured data in JSONB columns, write custom queries, and apply `@Transactional` to keep multi-step operations atomic.

1 Project Setup

The starter project contains a Spring Boot skeleton with the required Maven dependencies already configured. Your job is to start the database and verify the connection.

Clone the repo that has the base code to start working on this lab manual through this link `https://github.com/Scalable-2026/Lab04-Base-Code.git`

a) Open a terminal in the project root directory and start PostgreSQL:

```
docker compose up -d
```

Verify the container is running:

```
docker ps
```

You should see a `postgres` container with port `5432` mapped.

b) Open `src/main/resources/application.properties` and confirm the following values match the `docker-compose.yml`:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/lab04db
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

`ddl-auto=update` tells Hibernate to create or alter tables automatically based on your entity classes. `show-sql=true` prints every generated SQL statement to the console — you will use this throughout the lab to verify what Hibernate does behind the scenes.

2 JPA Entity and Basic CRUD

Every table in the database corresponds to a Java class annotated with `@Entity`. In this section you create the `Product` entity and get full CRUD operations for free via `JpaRepository`.

a) Open the `Product` class in the `model` package. Add the JPA annotations to map it to a database table:

```
@Entity
@Table(name = "products")
public class Product {
```

```java
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private String category;

    @Column(nullable = false)
    private Double price;

    @Column(name = "stock_qty", nullable = false)
    private Integer stockQuantity;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }
}
```

Key points: `@Id` + `@GeneratedValue(IDENTITY)` makes the column an auto-incrementing primary key. `@PrePersist` runs `onCreate()` automatically before every INSERT — you never call it manually. `updatable = false` on `createdAt` ensures Hibernate never overwrites it on UPDATE.

b) Open `ProductRepository` in the `repository` package. It should extend `JpaRepository`:

```java
@Repository
public interface ProductRepository
        extends JpaRepository<Product, Long> {
}
```

This empty interface gives you `save()`, `findById()`, `findAll()`, `deleteById()`, `count()`, and `existsById()` — zero implementation code required.

c) Open `ProductService` and implement the CRUD methods by delegating to the repository. Each method is marked with a `// TODO` comment describing what to write. For example, `createProduct` should call `productRepository.save(product)` and return the saved entity.

d) Open `ProductController` and Uncomment it.

3 Table Relationships — Customers, Orders, and Order Items

Storing all data in one table leads to duplication. When Fatima places 100 orders, her name and email should not be copied into 100 rows. Instead, the `orders` table stores a `customer_id` — a foreign key that points to one row in `customers`. JPA models this with `@ManyToOne` and `@OneToMany`.

3.1 Customer and Order Entities

a) Open the `Order` class.[1] Add the owning side of the relationship:

[1] The table is named `orders` (plural) because `order` is a reserved SQL keyword in PostgreSQL.

```java
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "customer_id", nullable = false)
    private Customer customer;

    @Column(name = "order_date", updatable = false)
    private LocalDateTime orderDate;

    @PrePersist
    protected void onCreate() {
        this.orderDate = LocalDateTime.now();
    }
}
```

`@ManyToOne` + `@JoinColumn` means: "this entity owns the foreign key column `customer_id`." Setting `order.setCustomer(c)` is what physically writes `customer_id = c.id` into the database row.

b) Open the `Customer` class in the `model` package and add the entity mapping:

```java
@Entity
@Table(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @OneToMany(mappedBy = "customer",
               cascade = CascadeType.ALL)
    @JsonIgnore
    private List<Order> orders = new ArrayList<>();
}
```

`mappedBy = "customer"` tells JPA: "do not create a foreign key column here — the `Order` entity already owns the relationship via its `customer` field." The value `"customer"` is the **Java field name** in `Order`, not a column name.[2]

c) Create `CustomerRepository` extending `JpaRepository<Customer, Long>`.

d) Wire through a `CustomerService` the same way you did for products and Uncomment the CustomerController.

[2] A common mistake is writing `mappedBy = "customer_id"` (the column name). JPA cannot find a field by that name and throws an exception at startup.

3.2 Understanding Relationship Ownership (You can try it on your own by creating a dummy route and call it to see the Hibernate Output)

The `Order` entity holds the foreign key — it is the **owner**. The `Customer` side is read-only navigation. This asymmetry matters when saving.

a) **Wrong way — setting from the non-owner side only:**

```
Customer c = customerRepository.findById(1L).get();
Order o = new Order();
c.getOrders().add(o); // non-owner side
orderRepository.save(o);
```

Check the `orders` table in the console SQL output. The `customer_id` column is `NULL`. The non-owner side has no column — it cannot write to the database.

b) **Correct way — set from the owner side:**

```
Customer c = customerRepository.findById(1L).get();
Order o = new Order();
o.setCustomer(c);   // owner side -- writes the FK
orderRepository.save(o);
```

Check the SQL output again. This time `customer_id = 1` is present in the INSERT.

c) **Best practice — set both sides:**

```
o.setCustomer(c);      // writes FK to database
c.getOrders().add(o);  // keeps Java object in sync
```

The second line does not persist anything — it only prevents `c.getOrders()` from returning a stale list within the same transaction.

3.3 The Join Entity — OrderItem

An order contains products. Each line has a `quantity` and a `unitPrice` — data that belongs to the *relationship*, not to the Order or the Product alone. `@ManyToMany` cannot store extra columns. The solution is a **Join Entity**: a normal `@Entity` with two `@ManyToOne` fields plus the extra data.

a) Open the `OrderItem` class and add the mapping:

```
@Entity
@Table(name = "order_items")
public class OrderItem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;

    @ManyToOne
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(nullable = false)
    private Integer quantity;
```

```
    @Column(name = "unit_price", nullable = false)
    private Double unitPrice;
}
```

Both fields are `@ManyToOne` because: one `OrderItem` links to exactly one `Order`, but one `Order` can have many `OrderItems`. The same logic applies to `Product`.

b) Create `OrderItemRepository` extending `JpaRepository<OrderItem, Long>` and Uncomment ***OrderRepository***.

c) Now build the order placement flow. Open `OrderService` and implement the `placeOrder` method. The starter code has the following TODO:

```
// TODO: Implement placeOrder [3]
// 1. Look up the customer by id (throw if not found)
// 2. Create a new Order, set the customer (owner side)
// 3. For each item in the request:
//    a. Look up the product (throw if not found)
//    b. Check stock >= requested quantity
//    c. Deduct stock and save the product
//    d. Create an OrderItem, set order + product +
// 4. Save the order (cascade saves all OrderItems)
// 5. Return the saved order
```

3.4 Run your application

a) Run the application and then open a browser or Postman and hit `http://localhost:8080/api/health`. You should receive:

```
OK
```

The project is now connected to PostgreSQL and ready for development.

b) **Test your work.** Start the application and use Postman or curl:

```
# Create a product
POST http://localhost:8080/api/products
'{"name":"Laptop","category":"ELECTRONICS",
      "price":1200.00,"stockQuantity":50}'

# Get all products
http://localhost:8080/api/products

# Get one product
http://localhost:8080/api/products/1

# Update a product
PUT http://localhost:8080/api/products/1
'{"name":"Laptop Pro","category":"ELECTRONICS",
      "price":1500.00,"stockQuantity":45}'

# Delete a product
DELETE http://localhost:8080/api/products/1
```

Check the IntelliJ console after each request — `show-sql=true` prints the exact SQL Hibernate generated.

[3] Do not add `@Transactional` yet — we will revisit this method in Section 6 to see what breaks without it.

```
List<Product> findByCategory(String category);
```

Spring parses the method name and generates `SELECT * FROM products WHERE category = ?` automatically. Expose it via `GET /api/products/category/{category}` in the controller (Uncomment its part in the Controller and the Service) and test with:

```
http://localhost:8080/api/products/category/ELECTRONICS
```

c) Test by creating one customer:

```
'{"name":"Fatima Ali","email":"fatima@example.com"}'
```

d) Expose `POST /api/orders` in `OrderController`. The request body should look like:

```
{
  "customerId": 1,
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 2, "quantity": 1 }
  ]
}
```

4 JSONB — Storing Semi-Structured Product Attributes

a) Open the `Product` entity and add a JSONB field:

```
@JdbcTypeCode(SqlTypes.JSON)
@Column(columnDefinition = "jsonb")
private Map<String, Object> attributes = new HashMap<>();
```

Import `org.hibernate.annotations.JdbcTypeCode` and `org.hibernate.type.SqlTypes`. Hibernate automatically serialises the Map to a JSON string on save and deserialises it back to a Map on load — no manual converter needed in Spring Boot 3.x.@[4]

b) Add this dependency to your pom.xml file as there is an internal bug inside Hibernate that requires us to use an older version of Jackson to serialize JSONB objects with Hibernate:

```
<!-- Hibernate 7.2 still needs Jackson 2.x ObjectMapper for JSONB FormatMapper -->
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

c) Update your `POST /api/products` endpoint to accept attributes. The request body should now support:

```
{
  "name": "iPhone 15",
  "category": "ELECTRONICS",
  "price": 999.00,
  "stockQuantity": 100,
  "attributes": {
    "brand": "Apple",
```

[4] In Spring Boot 2.x you would need a custom `AttributeConverter`. Spring Boot 3.x with Hibernate 6 handles JSONB natively via `@JdbcTypeCode`.

```
    "warranty_months": 12,
    "color": "Black"
  }
}
```

Insert at least three products with varying attributes:

- An electronics product with `brand`, `warranty_months`
- A clothing product with `material`, `sizes_available`
- A product with no attributes (empty map)

```
@Query(value =
    "SELECT * FROM products " +
    "WHERE attributes @> CAST(:filter AS jsonb)",
    nativeQuery = true)
List<Product> findByAttribute(
    @Param("filter") String filter);
```

The `@>` operator checks whether the JSONB column contains the given key-value pair. The parameter `filter` is a JSON string, e.g. `"{"brand":"Apple"}"`.[5]

d) In `ProductService`, add a method that builds the filter string from the key and value parameters:

```
public List<Product> searchByAttribute(
        String key, String value) {
    String filter = "{\""+key+"\":\""+value+"\"}"
    return productRepository.findByAttribute(filter);
}
```

e) Expose `GET /api/products/search?attributeKey=brand&attributeValue=Apple` in the controller and test:

```
curl "http://localhost:8080/api/products/search?\
attributeKey=brand&attributeValue=Apple"
```

Only products whose `attributes` column contains `{"brand":"Apple"}` should be returned.

5 Custom Queries and Service Layer Orchestration

`JpaRepository` auto-generates simple lookups, but production applications need joins, aggregations, and multi-source business logic. This section introduces two tools: `@Query` for writing the exact SQL or JPQL you need, and the **Service layer** for applying business rules on top of what the database returns.

5.1 JPQL Queries with @Query

a) Add a JPQL query to `ProductRepository` that finds products with stock below a threshold:

```
@Query("SELECT p FROM Product p WHERE p.stockQuantity < :threshold")
List<Product> findLowStock(@Param("threshold") int threshold);
```

JPQL uses the **Java class name** (`Product`) and **field names** (`p.stockQuantity`), not the SQL table or column names. Hibernate translates to PostgreSQL SQL automatically.[6]

[5] JPQL cannot express JSONB operators — this is one of the cases where `nativeQuery = true` is required.

[6] If you accidentally write the table name (`products`) instead of the class name (`Product`), Hibernate throws an error at startup — it validates JPQL at boot time, unlike native SQL which only fails at runtime.

b) Add a JPQL query to `OrderItemRepository` that returns the total units sold per product since a given date:

```
@Query("SELECT oi.product.id, SUM(oi.quantity) " +
       "FROM OrderItem oi " +
       "WHERE oi.order.orderDate >= :since " +
       "GROUP BY oi.product.id")
List<Object[]> findUnitsSoldSince(
    @Param("since") LocalDateTime since);
```

Each row in the result is an `Object[]` where index `[0]` is the product id and index `[1]` is the total quantity. This is a standard JPQL aggregation pattern.

5.2 DTO Projections

Loading full entities for read-only list endpoints is wasteful — Hibernate tracks every field for dirty-checking even if you never update anything. A **DTO projection** fetches only the columns you need and skips all Hibernate overhead.

a) Create a Java record in the `dto` package:

```
public record OrderSummaryDTO(
    Long orderId,
    String customerName,
    int itemCount
) {}
```

Records auto-generate the constructor, getters, `equals()`, and `hashCode()`.

b) Add a JPQL constructor expression query to `OrderRepository`:[7]

```
@Query("SELECT new com.example.lab04.dto.OrderSummaryDTO(" +
       "o.id, o.customer.name, SIZE(o.items)) " +
       "FROM Order o")
List<OrderSummaryDTO> findAllSummaries();
```

The `new com.example.lab04.dto.OrderSummaryDTO(...)` syntax tells JPQL to call your record's constructor directly. The **full package path** is required — a typo causes a runtime error.@[8]

c) Expose `GET /api/orders/summary` in the controller and test. Verify in the console that Hibernate generates a single SELECT with only the three columns — no lazy-loading, no N+1.

5.3 Service Layer Orchestration — Low Stock Report

The repository fetches raw data. The service combines it and applies business logic. This separation is what makes your code testable without a database.

a) Create a DTO record for the report:

```
public record LowStockAlertDTO(
    Long productId,
    String name,
    int stockQty,
    long unitsSoldThisMonth,
    String urgency
```

[7] You will need to create `OrderRepository extends JpaRepository<Order, Long>` if you have not already.

[8] Adjust the package path to match your actual project package. If your base package is different from `com.example.lab04`, update accordingly.

```
) {}
```

b) Open `ProductService` and implement `getLowStockReport`. The starter code has the following TODO:

```
// TODO: Implement getLowStockReport(int threshold)
//
// 1. Call productRepository.findLowStock(threshold)
//    to get all products below the threshold.
//
// 2. Call orderItemRepository.findUnitsSoldSince(
//    LocalDateTime.now().minusDays(30))
//    to get sales data for the last 30 days.
//
// 3. Build a Map<Long, Long> from the Object[] results:
//    key = productId, value = totalUnitsSold.
//    Use a plain for-loop (not streams).
//
// 4. For each low-stock product, look up its sales
//    from the map (default to 0 if not found).
//    Classify urgency:
//      CRITICAL = stock < 5 AND unitsSold > 50
//      HIGH     = stock < 10 AND unitsSold > 20
//      MEDIUM   = everything else
//
// 5. Sort the results: CRITICAL first, then HIGH,
//    then MEDIUM. Within the same urgency, sort by
//    stockQty ascending (lowest stock first).
//
// 6. Return List<LowStockAlertDTO>.
```

Follow the steps exactly. The urgency formula and sorting both happen in Java — not SQL. This is the key insight: business rules belong in the service layer where they are readable, changeable, and testable without a database.

c) Expose `GET /api/products/admin/low-stock?threshold=20` in the controller:

```java
@GetMapping("/admin/low-stock")
public ResponseEntity<List<LowStockAlertDTO>> getLowStock(
        @RequestParam(defaultValue = "20") int threshold) {
    return ResponseEntity.ok(
        productService.getLowStockReport(threshold));
}
```

6 Transactions and @Transactional (You can do it on your own to simulate the problem and solve it)

In Section 3 you built the `placeOrder` method without `@Transactional`. That method performs multiple database writes: deducting stock, saving an order, and saving order items. If the application crashes between any two of those writes, the database is left in an inconsistent state — stock deducted but no order created. `@Transactional` makes the entire method atomic: all writes succeed together, or all are rolled back.

6.1 Observing the Problem

a) Open `OrderService.placeOrder`. Confirm it does **not** have `@Transactional`.

```
product.setStockQuantity(
    product.getStockQuantity() - item.getQuantity());
productRepository.save(product);

// Simulate a crash after stock deduction
if (true) {
    throw new RuntimeException("Simulated server crash");
}
```

b) Place an order via `POST /api/orders`. The request will fail with a 500 error.

c) Check the database state. Call `GET /api/products/{id}` for the product whose stock you deducted. The stock is **permanently reduced** — but no order was created. This is a **partial write**: the database is now inconsistent.

d) Record this observation in `ANSWERS.md`: what was the stock before the request, what is it after, and how many orders exist for this customer?

6.2 Fixing It with @Transactional

a) Add `@Transactional` to the `placeOrder` method:

```
@Transactional
public Order placeOrder(OrderRequest request) {
    // ... same code as before ...
}
```

Spring wraps the method in a database transaction automatically. When the method returns normally, all changes are committed together. When an exception is thrown, all changes are rolled back — the stock deduction is undone.

b) **Reset the test data.** Restore the product's stock to its original value (use a PUT request or restart with fresh data).

c) Place the same order again. The request still fails with a 500 error (the `RuntimeException` is still there), but now check the product's stock. It should be **unchanged** — the rollback restored it.

d) Remove the simulated crash line (`if (true) throw ...`). Place a normal order and verify it succeeds end-to-end.

6.3 Seeding the Database with Sample Data

The starter project includes a `DataSeeder` class in the `config` package. It inserts 8 products (with varying stock levels and JSONB attributes), 3 customers, and 5 orders with order items. The seeder only runs when the **seed** profile is active, so it will not interfere with your normal development workflow.

Run the seeder once after completing Sections 2 and 3 (once your entities compile and the tables exist). Stop the application, then restart it with the seed profile:

```
mvn spring-boot:run -Dspring-boot.run.profiles=seed
```

Alternatively, in IntelliJ: go to **Run** → **Edit Configurations**, and add `seed` to the **Active Profiles** field. Then run normally.

You should see in the console:

```
DataSeeder: inserting sample data...
  8 products inserted
  3 customers inserted
```

```
  5 orders inserted
DataSeeder: done.
```

The seeder is idempotent — if data already exists it prints a skip message and does nothing. After seeding, **remove** the `seed` profile (or just restart without it) so it does not run on every boot.

The seeded data is designed to make Sections 4, 5, and 6 immediately testable:

- Products span multiple categories (`ELECTRONICS`, `CLOTHING`, `GROCERY`, `STATIONERY`) with varied JSONB attributes — useful for Section 4 JSONB queries.
- Stock levels range from 2 to 200, and existing orders create sales history — useful for the Section 5 low-stock report.
- Multiple customers with multiple orders — useful for Section 3 N+1 observation and Section 5 DTO projections.

7 Lab Solution

You can clone this repo `https://github.com/Scalable-2026/Lab-04-Soluition.git` to access the solution of the lab. You will also find a postman collection that you can import in postman and try the different routes.